Homework 6: Hardware and Software (15 points)

Due to Gradescope by 11:45 PM on Tuesday, October 4th
You need to submit a pdf to Gradescope; failure to assign questions to pages will result in a 10% deduction on your grade.

Homework Goal: Understand memory hierarchy and access;
work with assembly language and machine language

Hardware

1. Practice your vocab from these chapters! (1 point)
   Memory is divided into fixed-size units called cells, each of which has a standard size of 1 byte.
   Cache stores info from a slower device to be accessed faster, while registers store values that are being used during processing.

2. What are the three components that formally define a computer? (We covered this during lecture on 9/21) (1 point)

   1) Takes input
   2) Processes input
   3) Produces output

3. What are the **four** key subsystems of von Neumann architecture, and what is the purpose of each subsystem in a few words? (1 point)

   I/O : Communicate with keyboard, screen, etc
   Control Unit: Execute program, instructions, CPU cycle
   Arithmetic Logic Unit: logic operations (+, -, /, *, <, >, =)
   Memory: Store data

4. Describe the two key issues with the von Neumann architecture covered in class. (1 point)

   1) Processor growth rates slowing
   2) Memory access is slow

5. Perform the following conversions. *You must show your work for full credit. Recall that you should always use 2^n for KiB, MiB, GiB, etc.* (2 points)

   (a) 56,320 bytes to KiB

   $56{,}320\ B \left(\frac{KiB}{2^{10}\ B}\right) = 55.01\ KiB$

   (b) 100,663,296 *bits* to MiB

   $100{,}663{,}296\ b \left(\frac{1\ B}{8\ b}\right)\left(\frac{MiB}{2^{20}\ B}\right) = 12\ MiB$

   (c) 67,108,864 KiB to GiB

   $67{,}108{,}864\ KiB \left(\frac{2^{10}\ B}{1\ KiB}\right)\left(\frac{1\ GiB}{2^{30}\ B}\right) = 64\ GiB$

   (d) 12 MiB to KiB

   $12\ MiB \left(\frac{2^{20}\ B}{1\ MiB}\right)\left(\frac{1\ KiB}{2^{10}\ B}\right) = 12{,}288\ KiB$

6. To have 8 memory addresses where each memory address holds one byte of data (byte-addressable), we need 3 bits to index every address. If you have 16 GiB of RAM memory, then how many bits are needed to have memory addresses for every byte-addressable memory location? (1 point)

   $\frac{2^{30}}{1\ GiB} \cdot \frac{2^{4}}{B} = 34\ Bits$

7. What is something that can be done to increase the cache hit rate? (1 point)

   Be smart about populating/refreshing cache data and increase cache size

8. Consider the following structure of the instruction register. (1.5 points)

Opcode	Address Field 1	Address Field 2	Address Field 3
5 bits	9 bits	9 bits	9 bits

(a) What is the maximum number of distinct operations that can be recognized and executed by the processor on this machine? (Hint: What part of the instruction determines what the operation is?)

$2^5 = 32$ different op codes

(b) What is the maximum memory size on this machine? (Hint: Each memory location has an address.) *Assume each memory cell is one byte.*

$2^9 = 512B$

(c) How many bytes are required for each instruction?

at a minimum $2^{14} = 16,384B$

Software

9. Rank the following programming languages from the lowest level (left) to the highest level (right): ~~Python~~, ~~Machine language~~, ~~Assembly language~~ (0.5 points)

Machine language, Assembly, Python

10. What is the difference between a compiler and an interpreter? **Give an example** of a language that uses each. (1 point)

A compiler takes a programming language and outputs machine code (C++, C)

An interpreter translates one statement at a time (Python)

11. Assume memory address M[1] contains the value 2, address M[2] contains the value 4, and address M[3] contains the value 6. What are the values of all three addresses after the following assembly instructions are executed? SUB M[3] M[2] M[1] works how you might expect (i.e., similar to ADD M[3] M[2] M[1]) (1 point)

```
ADD M[1] M[2] M[1]   M[1] = 4+2 = 6
SUB M[3] M[3] M[1]   M[3] = 6-6 = 0
ADD M[1] M[1] M[1]   M[1] = 6+6 = 12
SUB M[1] M[2] M[3]   M[1] = 4-0 = 4
ADD M[3] M[3] M[2]   M[3] = 0+4 = 4
SUB M[2] M[2] M[3]   M[2] = 4-4 = 0
```

```
Final value of M[1]: 4
Final value of M[2]: 0
Final value of M[3]: 4
```

a ↑ b ↑ c ↑ d ↑

12. Suppose a, b, c, and d are in memory locations M[100], M[101], M[110], and M[111], respectively. Based on the following assembly language instructions, write an algebraic equation for the *final* value of c using the *original* values of a, b, c, and d. (1 point)

```
ADD M[100] M[100] M[100]   a = a+a
ADD M[100] M[100] M[100]   a = a+a
ADD M[111] M[111] M[111]   d = d+d
ADD M[110] M[100] M[111]   c = 4a + 2d
ADD M[110] M[110] M[101]   c = (4a+2d) + b
```

Equation:

$C = (4a + 2d) + b$

13. Assume the variables v, w, x, y, and z are stored in memory locations M[001], M[010], M[011], M[100], and M[101], respectively. Using the instruction set below, fill in the blanks to translate the following pseudocode into their machine language equivalents. Memory location M[110] is available for any intermediate calculations. (2 points)

M[001] = V
M[010] = W
M[011] = X
M[100] = Y
M[101] = Z

Opcode (3 bits)	Operation
000	Stop
001	Input
010	Output
011	Add
100	Mult

a. Set v = (w + x) * (y + z) without modifying the contents of w, x, y, or z

add
0 1 1 0 0 1 V = 0 1 0 W + 0 1 1 X
add
0 1 1 1 1 0 A = 1 0 0 Y + 1 0 1 Z
multi
1 0 0 0 0 1 V = 0 0 1 V * 1 1 0 A
stop
0 0 0 0 0 0 0 0 0 0 0 0

b. Input v from the user, then display v * 2

(Recall, we can only represent addresses, not values, in our address fields)

input
0 0 1 0 0 1 0 0 0 0 0 0
mult
1 0 0 0 0 1 0 0 1 0 0 1
output
0 1 0 0 0 1 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0